Centro Federal de Educação Tecnológica de Minas Gerais
Professora: Poliana Corrêa

RELATÓRIO PRÁTICA 9 - LAOC
CAMINHO DE DADOS DAS INSTRUÇÕES DE DESVIO

ALUNOS:
GIOVANNI ESTEVAM DOS SANTOS
JOÃO VITOR LOPES FAGUNDES

Belo Horizonte - MG
2025

1. Introdução

A **Prática 9 (Caminho de Dados das Instruções de Desvio)** teve como foco principal alterar o projeto da arquitetura MIPS para permitir a mudança no fluxo de execução do programa, integrando a lógica de saltos ao processador. O objetivo é expandir o caminho de dados para suportar instruções de desvio incondicional (**Jump - j**), do Formato-J, e desvio condicional (**Branch Equal - beq**), do Formato-I.

Para o funcionamento da instrução *Jump*, implementou-se o cálculo do endereço-alvo através da concatenação dos 4 bits mais significativos de (PC+4) com os 26 bits do campo imediato da instrução (após um *shift left 2*). Já para a instrução *beq*, a ULA passa a ser utilizada para subtrair os valores dos registradores e gerar o sinal **Zero**, que, combinado com o sinal de controle **Branch**, decide se o *Program Counter* (PC) deve ser atualizado com o endereço de desvio calculado. A Unidade de Controle foi devidamente ajustada para incluir o sinal **Jump** e gerenciar os multiplexadores que definem a nova fonte de dados para o PC.

2. Desenvolvimento

Nosso projeto da Prática 9 consiste na expansão final do caminho de dados do processador MIPS monociclo para suportar as instruções de controle de fluxo (Branch e Jump). O foco do projeto foi a implementação da lógica de decisão de desvio, cálculo de novos endereços para o PC e a atualização da Unidade de Controle para gerenciar essas transferências. A arquitetura agora é capaz de alterar o fluxo de execução do programa baseada em comparações (beq) ou saltos incondicionais (j), completando o conjunto básico de instruções MIPS.

Descrição e Decisões de Projeto dos Módulos

Unidade de Controle Principal: unidadeControlePrincipal

- **Descrição:** Este módulo decodifica o *opcode* da instrução e gera os sinais de controle para o datapath. Nesta prática, ele foi expandido para identificar as instruções beq (opcode 000100) e j (opcode 000010).
- **Decisões de Projeto:**
    - **Sinais de Controle de Desvio:** Foram adicionados dois novos sinais de saída: Branch e Jump. O sinal Branch é ativado quando a instrução é um *beq*, e o sinal Jump é ativado exclusivamente para a instrução *j*.
    - **Lógica Combinacional:** A lógica interna utiliza portas AND e NOT sobre os bits do opcode para garantir que apenas a instrução correta ative seus respectivos sinais de controle, evitando conflitos no datapath.

Processador (Integração e Lógica de PC): processador

- **Descrição:** O módulo de topo conecta todos os componentes e agora integra a lógica complexa de atualização do Program Counter (PC). É aqui que os endereços de desvio são calculados e selecionados.
- **Decisões de Projeto:**
    - **Cálculo do Endereço de Branch:** Foi instanciado um somador dedicado (somadorEnderecoBranch) que soma o valor de PC + 4 com o imediato estendido de sinal deslocado à esquerda em 2 bits (immediate_32bits << 2). Isso permite calcular o endereço relativo para o desvio condicional[3].
    - **Cálculo do Endereço de Jump:** O endereço de salto incondicional é formado pela concatenação dos 4 bits mais significativos de PC + 4 com os 26 bits de endereço da instrução (multiplicados por 4 via shift left 2). Essa lógica foi implementada via assign direto para criar o fio jumpAddress.

    - **Hierarquia de Multiplexadores do PC:** Para decidir o próximo valor do PC, utilizou-se uma estrutura de dois MUXes em cascata. O primeiro (muxControlBranch) decide entre o fluxo normal (PC + 4) e o endereço de *Branch*, controlado pela operação lógica Branch & zero (a porta AND que valida se a condição de igualdade foi satisfeita). O segundo (muxControlJump) decide entre o resultado anterior e o jumpAddress, controlado pelo sinal Jump, tendo prioridade final sobre o valor escrito no PC.

Memória de Instrução: memoriaInstrucao

- **Descrição:** Armazena o programa a ser executado. Para esta prática, o conteúdo foi alterado para conter um código específico que testa loops e saltos.
- **Decisões de Projeto:**
    - **Programa de Teste Hardcoded:** O bloco initial foi reescrito para conter o código de máquina solicitado no roteiro: beq, addi, sw, lw, sub, j. As instruções foram convertidas manualmente para hexadecimal byte a byte. Por exemplo, o beq $8, $9, 1 foi codificado nos endereços 0-3 e o j 0 no endereço 20 para criar um loop infinito reiniciando o programa.
    - **Endereçamento Byte a Byte:** Mantivemos a estrutura de array de 8 bits, onde cada instrução ocupa 4 posições de memória, concatenadas na saída para formar a palavra de 32 bits.

Banco de Registradores: bancoRegistradores

- **Descrição:** Gerencia a leitura e escrita nos 32 registradores de uso geral.
- **Decisões de Projeto:**
    - **Inicialização Específica para Teste:** A lógica de reset foi modificada para carregar valores específicos exigidos pelo roteiro de teste da Prática 9. Os registradores $8 e $9 são inicializados com o valor 5 (para testar a igualdade no *beq*), $10 com 1 e $12 com 4 (base para memória). Isso elimina a necessidade de instruções extras de addi apenas para configurar o estado inicial do teste.

Unidade de Controle da ULA e ULA32bits

- **Descrição:** Realiza as operações aritméticas e lógicas. Na instrução *beq*, a ULA desempenha um papel fundamental de comparação.
- **Decisões de Projeto:**
    - **Uso da Flag Zero:** Para a instrução beq, a Unidade de Controle da ULA configura a ULA para realizar uma subtração (sub). O resultado numérico é descartado, mas o sinal de saída zero (que se torna 1 se o

resultado da subtração for 0, ou seja, $A = B$) é enviado ao módulo processador para autorizar o desvio condicional.

Contador de Programa: countPC

- **Descrição:** Registrador que mantém o endereço da instrução atual.
- **Decisões de Projeto:**
    - **Entrada Flexível:** O módulo recebe o sinal pcin que agora vem da saída do multiplexador de Jump (muxControlJump), permitindo que o PC seja atualizado não apenas sequencialmente, mas para qualquer endereço calculado pela lógica de desvio.

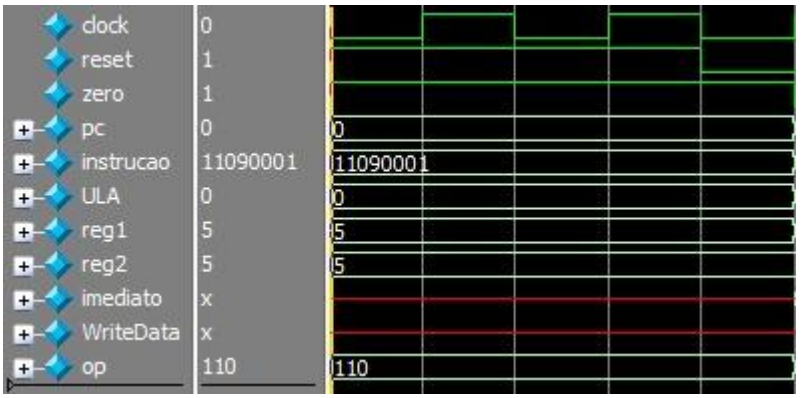
Memória de Dados: memoriaDados

- **Descrição:** Responsável por armazenar dados manipulados pelas instruções lw e sw.
- **Decisões de Projeto:**
    - **Integração no Fluxo de Teste:** Embora não tenha sofrido alterações estruturais nesta prática, sua funcionalidade é validada pelo programa de teste que executa um sw seguido de um lw nos endereços 8 e 12, garantindo que os desvios (Branch/Jump) não interfiram na consistência dos dados gravados e lidos da RAM.

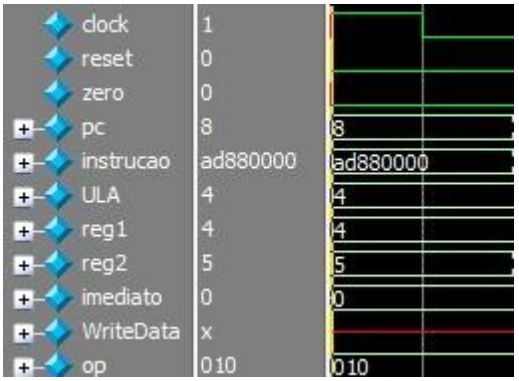
Top Level

- **Descrição:** Embora o código verilog do topo não tenha sido fornecido explicitamente, o módulo processador exporta todos os sinais necessários (pcout_out, zero_out, branch_out, etc.) para visualização.
- **Decisões de Projeto:**
    - **Visualização de Depuração:** As saídas do processador foram roteadas para permitir que, na FPGA, os displays mostrem o PC atual (verificando se o salto ocorreu), o bit zero da ULA (confirmando a comparação dos registradores) e os dados lidos/escritos, conforme solicitado no item III.c do roteiro. Isso permite validar visualmente se o programa está preso no loop correto ou se o desvio condicional falhou.

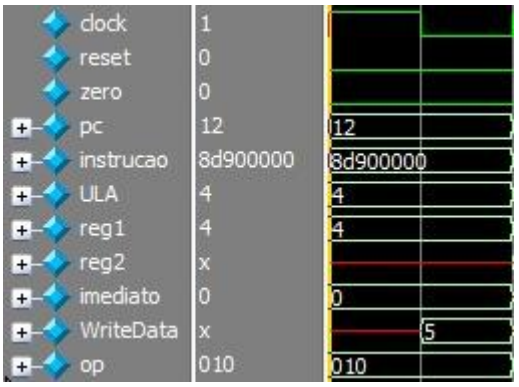
3. Simulações



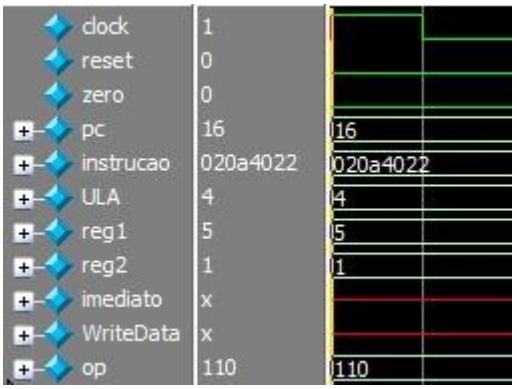
No primeiro ciclo de clock, pc = 0 está na primeira instrução beq $8, $9, 1. Como os registradores $8 e $9 foram ambos inicializados com 5, a operação beq faz com que a ULA realize uma subtração entre os dois registradores, o resultado entre elas é zero. Tal resultado faz com que a operação beq seja verdadeira e então, pule para a instrução PC = 4 + 1*4(shift2)=8.



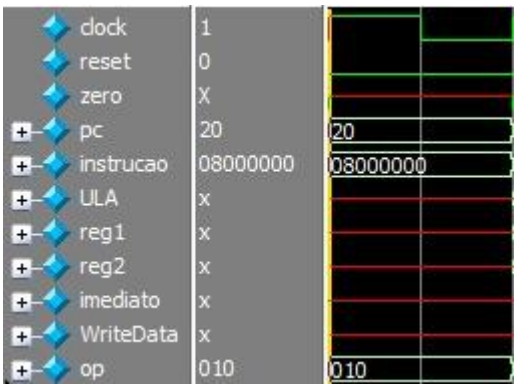
No segundo ciclo de clock, pc = 8, a instrução realizada é sw $8, 0($12), após o pulo condicional ser verdadeiro da instrução anterior. A ULA faz o cálculo do endereço $12 + 0 = 4 + 0 = 4 e armazena o valor 5 nele.



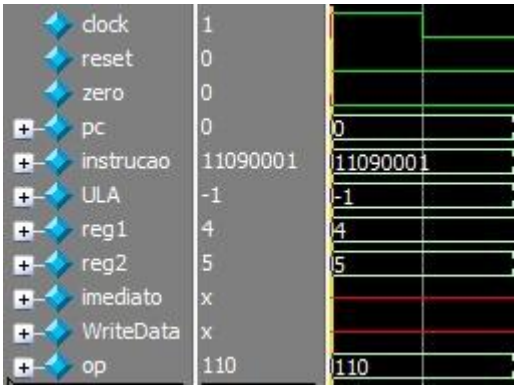
No terceiro ciclo de clock, pc = 12 está na instrução lw $16, 0($12). A ULA faz o cálculo do endereço $12 + 0 = 4 e salva o valor desse endereço no registrador $16.



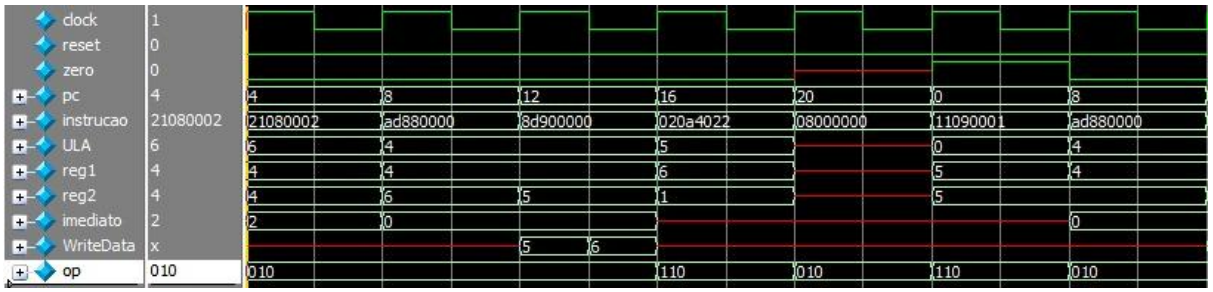
No terceiro ciclo de clock, pc = 12 está na instrução sub $8, $16, $10. A ULA faz o cálculo $16 - $10 = 5 - 1 = 4 e armazena em $8.



No quarto ciclo de clock, a instrução j 0 é executada e pc irá voltar para a primeira instrução.



No quinto ciclo de clock, pc = 0 e, novamente, beq $8, $9, 1 será executado. $8 - $9 = 4 - 5 = -1. Como o pulo condicional é falso, o fluxo sequencial segue normalmente.



No sexto ciclo de clock, pc = 4 e a instrução addi $8, $8, 2 é executada, fazendo $8 + 2 = 4 + 2 = 6. Em seguida, o código se repete em um loop infinito.